

1.4.事务和事务的隔离级别

1.4.1.为什么需要事务

事务是数据库管理系统（DBMS）执行过程中的一个逻辑单位（不可再进行分割），由一个有限的数据库操作序列构成（多个DML语句，select语句不包含事务），要不全部成功，要不全部不成功。

A 给B 要划钱，A 的账户-1000元，B 的账户就要+1000元，这两个update 语句必须作为一个整体来执行，不然A 扣钱了，B 没有加钱这种情况就是错误的。那么事务就可以保证A、B 账户的变动要么全部一起发生，要么全部一起不发生。

1.4.2.事务特性

事务应该具有4个属性：原子性、一致性、隔离性、持久性。这四个属性通常称为ACID特性。

I 原子性 (atomicity)

I 一致性 (consistency)

I 隔离性 (isolation)

I 持久性 (durability)

1.4.2.1.原子性 (atomicity)

一个事务必须被视为一个不可分割的最小单元，整个事务中的所有操作要么全部提交成功，要么全部失败，对于一个事务来说，不能只执行其中的一部分操作。比如：

连老师借给李老师1000元：

1.连老师工资卡扣除1000元

2.李老师工资卡增加1000元

整个事务的操作要么全部成功，要么全部失败，不能出现连老师工资卡扣除，但是李老师工资卡不增加的情况。如果原子性不能保证，就会很自然的出现一致性问题。

1.4.2.2.一致性 (consistency)

一致性是指事务将数据库从一种一致性转换到另外一种一致性状态，在事务开始之前和事务结束之后数据库中数据的完整性没有被破坏。

连老师借给李老师1000元：

1.连老师工资卡扣除1000元

2.李老师工资卡增加1000元

扣除的钱（-500）与增加的钱（500）相加应该为0，或者说连老师和李老师的账户的钱加起来，前后应该不变。

1.4.2.3.持久性 (durability)

一旦事务提交，则其所做的修改就会永久保存到数据库中。此时即使系统崩溃，已经提交的修改数据也不会丢失。

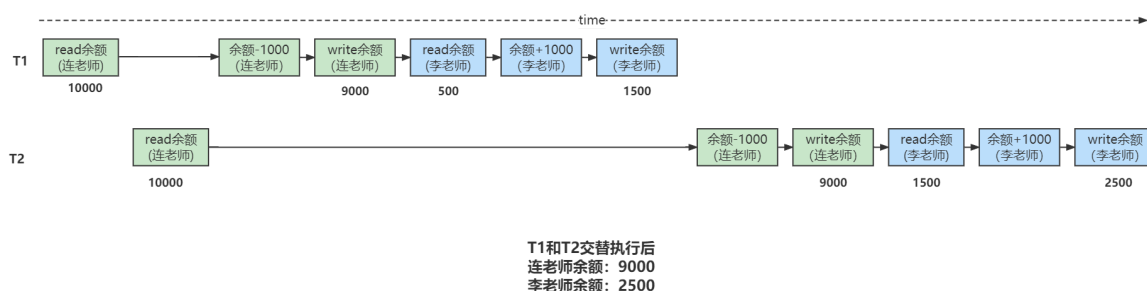
1.4.2.4.隔离性 (isolation)

一个事务的执行不能被其他事务干扰。即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。

如果隔离性不能保证，会导致什么问题？

连老师借给李老师生活费，借了两次，每次都是1000，连老师的卡里开始有10000，李老师的卡里开始有500，从理论上，借完后，连老师的卡里有8000，李老师的卡里应该有2500。

我们将连老师向李老师同时进行的两次转账操作分别称为T1和T2，在现实世界中T1和T2是应该没有关系的，可以先执行完T1，再执行T2，或者先执行完T2，再执行T1，结果都是一样的。但是很不幸，真实的数据库中T1和T2的操作可能交替执行的，执行顺序就有可能有是：



如果按照上图中的执行顺序来进行两次转账的话，最终我们看到，连老师的账户里还剩9000元钱，相当于只扣了1000元钱，但是李老师的账户里却成了2500元钱，多了10000元，这银行岂不是要亏死了？

所以对于现实世界中状态转换对应的某些数据库操作来说，不仅要保证这些操作以原子性的方式执行完成，而且要保证其它的状态转换不会影响到本次状态转换，这个规则被称之为隔离性。

1.4.3.事务并发引发的问题

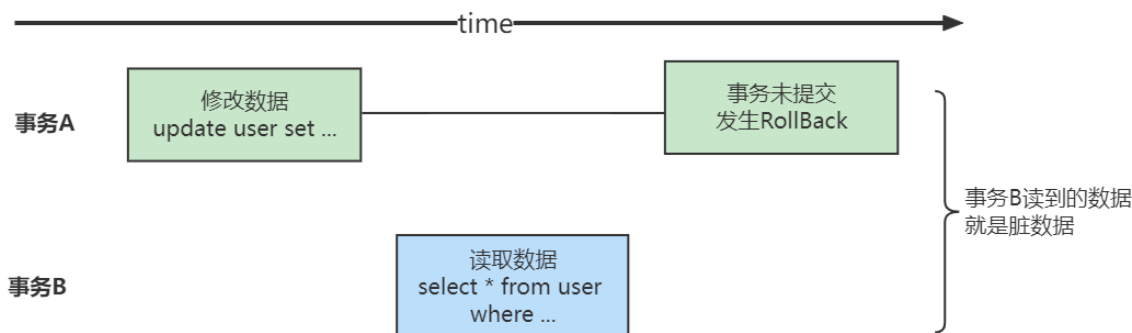
我们知道MySQL是一个客户端 / 服务器架构的软件，对于同一个服务器来说，可以有若干个客户端与之连接，每个客户端与服务器连接上之后，就可以称之为一个会话（Session）。每个客户端都可以在自己的会话中向服务器发出请求语句，一个请求语句可能是某个事务的一部分，也就是对于服务器来说可能同时处理多个事务。

在上面我们说过事务有一个称之为隔离性的特性，理论上在某个事务对某个数据进行访问时，其他事务应该进行排队，当该事务提交之后，其他事务才可以继续访问这个数据，这样的话并发事务的执行就变成了串行化执行。

但是对串行化执行性能影响太大，我们既想保持事务的一定的隔离性，又想让服务器在处理访问同一数据的多个事务时性能尽量高些，当我们舍弃隔离性的时候，可能会带来什么样的数据问题呢？

1.4.3.1.脏读

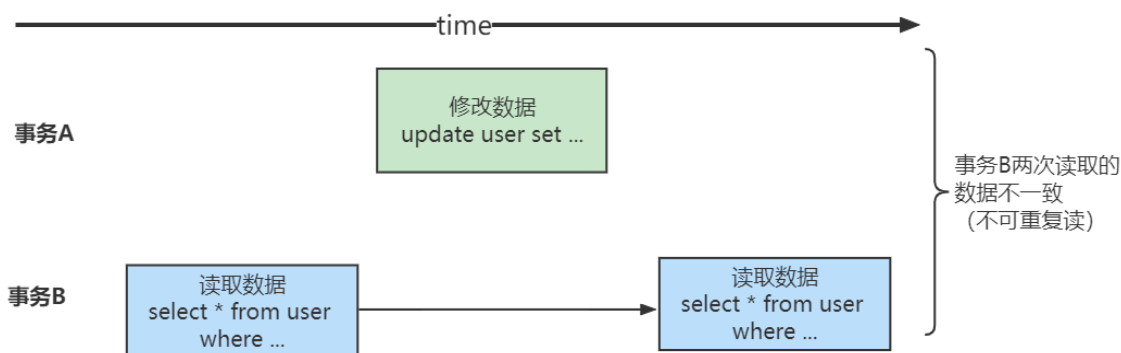
当一个事务读取到了另外一个事务修改但未提交的数据，被称为脏读。



- 1、在事务A执行过程中，事务A对数据资源进行了修改，事务B读取了事务A修改后的数据。
- 2、由于某些原因，事务A并没有完成提交，发生了RollBack操作，则事务B读取的数据就是脏数据。这种读取到另一个事务未提交的数据的现象就是脏读(Dirty Read)。

1.4.3.2.不可重复读

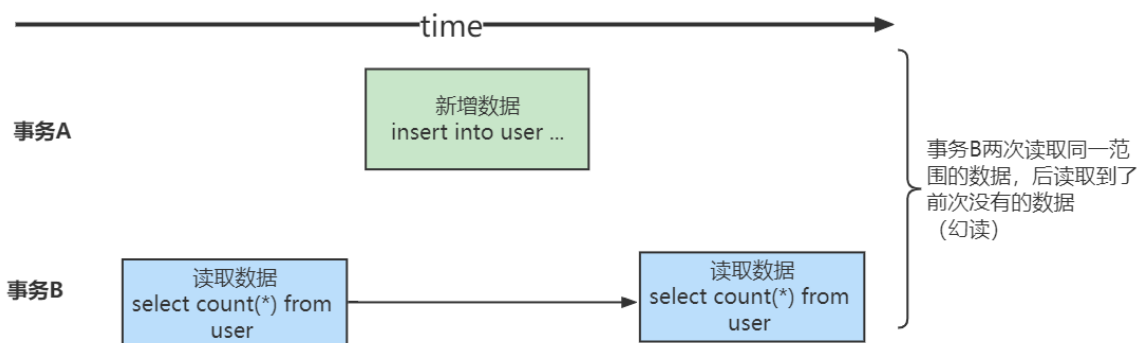
当事务内相同的记录被检索两次，且两次得到的结果不同时，此现象称为不可重复读。



事务B读取了两次数据资源，在这两次读取的过程中事务A修改了数据，导致事务B在这两次读取出来的数据不一致。

1.4.3.3.幻读

在事务执行过程中，另一个事务将新记录添加到正在读取的事务中时，会发生幻读。



事务B前后两次读取同一个范围的数据，在事务B两次读取的过程中事务A新增了数据，导致事务B后一次读取到前一次查询没有看到的行。

幻读和不可重复读有些类似，但是幻读重点强调了读取到了之前读取没有获取到的记录。

1.1.4.SQL标准中的四种隔离级别

我们上边介绍了几种并发事务执行过程中可能遇到的一些问题，这些问题也有轻重缓急之分，我们给这些问题按照严重性来排一下序：

脏读 > 不可重复读 > 幻读

我们上边所说的舍弃一部分隔离性来换取一部分性能在这里就体现在：设立一些隔离级别，隔离级别越低，越严重的问题就越可能发生。有一帮人（并不是设计MySQL的大叔们）制定了一个所谓的SQL标准，在标准中设立了4个隔离级别：

READ UNCOMMITTED：未提交读。

READ COMMITTED：已提交读。

REPEATABLE READ：可重复读。

SERIALIZABLE：可串行化。

SQL标准中规定，针对不同的隔离级别，并发事务可以发生不同严重程度的问题，具体情况如下：

也就是说：

READ UNCOMMITTED隔离级别下，可能发生脏读、不可重复读和幻读问题。

READ COMMITTED隔离级别下，可能发生不可重复读和幻读问题，但是不可以发生脏读问题。

REPEATABLE READ隔离级别下，可能发生幻读问题，但是不可以发生脏读和不可重复读的问题。

SERIALIZABLE隔离级别下，各种问题都不可以发生。

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	可能
SERIALIZABLE 可串行化	-	-	-

1.4.5.MySQL中的隔离级别

不同的数据库厂商对SQL标准中规定的四种隔离级别支持不一样，比方说Oracle就只支持READ COMMITTED和SERIALIZABLE隔离级别。本书中所讨论的MySQL虽然支持4种隔离级别，但与SQL标准中所规定的各级隔离级别允许发生的问题却有些出入，MySQL在REPEATABLE READ隔离级别下，是可以禁止幻读问题的发生的。

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	
SERIALIZABLE 可串行化	-	-	-

MySQL的默认隔离级别为REPEATABLE READ，我们可以手动修改事务的隔离级别。

1.4.5.1.如何设置事务的隔离级别

我们可以通过下边的语句修改事务的隔离级别：

SET [GLOBAL|SESSION] TRANSACTION ISOLATION LEVEL level;

其中的level可选值有4个：

```
level: {
    REPEATABLE READ
    | READ COMMITTED
    | READ UNCOMMITTED
    | SERIALIZABLE
}
```

设置事务的隔离级别的语句中，在SET关键字后可以放置GLOBAL关键字、SESSION关键字或者什么都不放，这样会对不同范围的事务产生不同的影响，具体如下：

使用GLOBAL关键字（在全局范围影响）：

比方说这样：

```
SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

则：只对执行完该语句之后产生的会话起作用。当前已经存在的会话无效。

使用SESSION关键字（在会话范围影响）：

比方说这样：

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

则：对当前会话的所有后续的事务有效

该语句可以在已经开启的事务中间执行，但不会影响当前正在执行的事务。

如果在事务之间执行，则对后续的事务有效。

上述两个关键字都不用（只对执行语句后的下一个事务产生影响）：

比方说这样：

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

则：只对当前会话中下一个即将开启的事务有效。下一个事务执行完后，后续事务将恢复到之前的隔离级别。该语句不能在已经开启的事务中间执行，会报错的。

如果我们在服务器启动时想改变事务的默认隔离级别，可以修改启动参数transaction-isolation的值，比方说我们在启动服务器时指定了--transaction-isolation=SERIALIZABLE，那么事务的默认隔离级别就从原来的REPEATABLE READ变成了SERIALIZABLE。

想要查看当前会话默认的隔离级别可以通过查看系统变量transaction_isolation的值来确定：

```
SHOW VARIABLES LIKE 'transaction_isolation';
```

```
mysql> SHOW VARIABLES LIKE 'transaction_isolation';
+-----+-----+
| Variable_name | Value          |
+-----+-----+
| transaction_isolation | REPEATABLE-READ |
+-----+-----+
1 row in set (0.00 sec)
```

或者使用更简便的写法：

```
SELECT @@transaction_isolation;
```

```
mysql> SELECT @@transaction_isolation;
+-----+
| @@transaction_isolation |
+-----+
| REPEATABLE-READ        |
+-----+
1 row in set (0.00 sec)
```

注意：transaction_isolation是在MySQL 5.7.20的版本中引入来替换tx_isolation的，如果你使用的是之前版本的MySQL，请将上述用到系统变量transaction_isolation的地方替换为tx_isolation。

1.4.6.MySQL事务

1.4.6.1.事务基本语法

事务开始

- 1、begin
- 2、START TRANSACTION（推荐）
- 3、begin work

事务回滚

rollback

事务提交

commit

使用事务插入两行数据，commit后数据还在

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into testdemo values(5,5,5);
Query OK, 1 row affected (0.00 sec)

mysql> insert into testdemo values(6,6,6);
Query OK, 1 row affected (0.00 sec)

mysql> select * from testdemo
-> ;
+----+-----+-----+
| id | order_id | goods_id |
+----+-----+-----+
| 5 | 5 | 5 |
| 6 | 6 | 6 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+----+-----+-----+
| id | order_id | goods_id |
+----+-----+-----+
| 5 | 5 | 5 |
| 6 | 6 | 6 |
+----+-----+-----+
2 rows in set (0.00 sec)
```

使用事务插入两行数据，rollback后数据没有了

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into testdemo values(7,7,7);
Query OK, 1 row affected (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+-----+-----+-----+
| id | order_id | goods_id |
+-----+-----+-----+
| 5 | 5 | 5 |
| 6 | 6 | 6 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

1.4.6.2.保存点

如果你开启了一个事务，执行了很多语句，忽然发现某条语句有点问题，你只好使用ROLLBACK语句来让数据库状态恢复到事务执行之前的样子，然后一切从头再来，但是可能根据业务和数据的变化，不需要全部回滚。所以MySQL里提出了一个保存点（英文：savepoint）的概念，就是在事务对应的数据库语句中打几个点，我们在调用ROLLBACK语句时可以指定会滚到哪个点，而不是回到最初的原点。定义保存点的语法如下：

SAVEPOINT 保存点名称;

当我们想回滚到某个保存点时，可以使用下边这个语句（下边语句中的单词WORK和SAVEPOINT是可有可无的）：

```
ROLLBACK TO [SAVEPOINT] 保存点名称;
```

不过如果ROLLBACK语句后边不跟随保存点名称的话，会直接回滚到事务执行之前的状态。

如果我们想删除某个保存点，可以使用这个语句：

```
RELEASE SAVEPOINT 保存点名称;
```



```
mysql> set autocommit=0;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like '%autocommit%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| autocommit    | OFF   |
+-----+-----+
1 row in set (0.01 sec)
```

```
mysql> insert into testdemo values(5,5,5);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> savepoint s1;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> insert into testdemo values(6,6,6);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> savepoint s2;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> insert into testdemo values(7,7,7);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> savepoint s3;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select * from testdemo
-> ;
```

id	order_id	goods_id
5	5	5
6	6	6
7	7	7

```
mysql> rollback to savepoint s2;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+-----+-----+-----+
| id | order_id | goods_id |
+-----+-----+-----+
| 5 | 5 | 5 |
| 6 | 6 | 6 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```



1.4.6.3.隐式提交

当我们使用START TRANSACTION或者BEGIN语句开启了一个事务，或者把系统变量autocommit的值设置为OFF时，事务就不会进行自动提交，但是如果我们输入了某些语句之后就会悄悄的提交掉，就像我们输入了COMMIT语句了一样，这种因为某些特殊的语句而导致事务提交的情况称为隐式提交，这些会导致事务隐式提交的语句包括：

1.4.6.3.1.执行DDL

定义或修改数据库对象的数据定义语言（Datadefinition language，缩写为：DDL）。

所谓的数据库对象，指的就是数据库、表、视图、存储过程等等这些东西。当我们使用CREATE、ALTER、DROP等语句去修改这些所谓的数据库对象时，就会隐式的提交前边语句所属于的事务，就像这样：

```
BEGIN;

SELECT ... # 事务中的一条语句

UPDATE ... # 事务中的一条语句

... # 事务中的其它语句

CREATE TABLE ...
```

```
mysql> select * from testdemo;
```

id	order_id	goods_id
5	5	5
6	6	6

2 rows in set (0.00 sec)

```
mysql> set autocommit=0;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> BEGIN;
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> update testdemo set goods_id =666 where id =6;
```

Query OK, 1 row affected (0.00 sec)

Rows matched: 1 Changed: 1 Warnings: 0

```
mysql> CREATE TABLE `testdemo_1` (
```

```
-> `id` int(11) NOT NULL,
```

```
-> `order_id` int(11) NOT NULL,
```

```
-> `goods_id` int(11) NOT NULL,
```

```
-> PRIMARY KEY (`id`)
```

```
-> ) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> select * from testdemo;
```

id	order_id	goods_id
5	5	5
6	6	666

```
mysql> select * from testdemo;
+----+-----+-----+
| id | order_id | goods_id |
+----+-----+-----+
| 5  |        5 |        5 |
| 6  |        6 |       666 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+----+-----+-----+
| id | order_id | goods_id |
+----+-----+-----+
| 5  |        5 |        5 |
| 6  |        6 |       666 |
+----+-----+-----+
2 rows in set (0.00 sec)
```

此语句会隐式的提交前边语句所属于的事务

1.4.6.3.2.隐式使用或修改mysql数据库中的表

当我们使用ALTER USER、CREATE USER、DROP USER、GRANT、RENAME USER、REVOKE、SET PASSWORD等语句时也会隐式的提交前边语句所属于的事务。

1.4.6.3.3.事务控制或关于锁定的语句

当我们在一个会话里，一个事务还没提交或者回滚时就又使用START TRANSACTION或者BEGIN语句开启了另一个事务时，会隐式的提交上一个事务，比如这样：

```
BEGIN;

SELECT ... # 事务中的一条语句

UPDATE ... # 事务中的一条语句

... # 事务中的其它语句

BEGIN; # 此语句会隐式的提交前边语句所属于的事务
```

```
mysql> select * from testdemo;
+----+-----+-----+
| id | order_id | goods_id |
+----+-----+-----+
| 5 |      5 |      5 |
| 6 |      6 |     666 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> update testdemo set goods_id =6 where id =6;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+----+-----+-----+
| id | order_id | goods_id |
+----+-----+-----+
| 5 |      5 |      5 |
| 6 |      6 |      6 |
+----+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+----+-----+-----+
| id | order_id | goods_id |
+----+-----+-----+
| 5 |      5 |      5 |
| 6 |      6 |      6 |
+----+-----+-----+
2 rows in set (0.00 sec)
```

或者当前的autocommit系统变量的值为OFF，我们手动把它调为ON时，也会隐式的提交前边语句所属的事务。

或者使用LOCK TABLES、UNLOCK TABLES等关于锁定的语句也会隐式的提交前边语句所属的事务。

1.4.6.3.4.加载数据的语句

比如我们使用LOAD DATA语句来批量往数据库中导入数据时，也会隐式的提交前边语句所属的事务。

1.4.6.3.5.关于MySQL复制的一些语句

使用START SLAVE、STOP SLAVE、RESET SLAVE、CHANGE MASTER TO等语句时也会隐式的提交前边语句所属的事务。

1.4.6.3.6.其它的一些语句

使用ANALYZE TABLE、CACHE INDEX、CHECK TABLE、FLUSH、LOAD INDEX INTO CACHE、OPTIMIZE TABLE、REPAIR TABLE、RESET等语句也会隐式的提交前边语句所属的事务。

1.5. MVCC-多版本并发控制

全称Multi-Version Concurrency Control，即多版本并发控制，主要是为了提高数据库的并发性能。同一行数据平时发生读写请求时，会上锁阻塞住。但MVCC用更好的方式去处理读—写请求，做到在发生读—写请求冲突时不用加锁。

这个读是指的快照读，而不是当前读，当前读是一种加锁操作，是悲观锁。

那它到底是怎么做到读—写不用加锁的，快照读和当前读是指什么？我们后面都会学到。

1.5.1.MVCC原理

1.5.1.1.复习事务隔离级别

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	
SERIALIZABLE 可串行化	-	-	-

MySQL在REPEATABLE READ隔离级别下，是可以很大程度避免幻读问题的发生的（好像解决了，但是又没完全解决），MySQL是怎么做到的？

1.5.1.2.版本链

必须要知道的概念（每个版本链针对的一条数据）：

我们知道，对于使用InnoDB存储引擎的表来说，它的聚簇索引记录中都包含两个必要的隐藏列（row_id并不是必要的，我们创建的表中有主键或者非NULL的UNIQUE键时都不会包含row_id列）：

trx_id：每次一个事务对某条聚簇索引记录进行改动时，都会把该事务的事务id赋值给trx_id隐藏列。

roll_pointer：每次对某条聚簇索引记录进行改动时，都会把旧的版本写入到undo日志中，然后这个隐藏列就相当于一个指针，可以通过它来找到该记录修改前的信息。

(补充点：undo日志：为了实现事务的原子性，InnoDB存储引擎在实际进行增、删、改一条记录时，都需要先把对应的undo日志记下来。**一般每对一条记录做一次改动，就对应着一条undo日志**，但在某些更新记录的操作中，也可能会对应着2条undo日志。一个事务在执行过程中可能新增、删除、更新若干条记录，也就是说需要记录很多条对应的undo日志，这些undo日志会被从0开始编号，也就是说根据生成的顺序分别被称为第0号undo日志、第1号undo日志、...、第n号undo日志等，这个编号也被称之为undo no。)

为了说明这个问题，我们创建一个演示表

```
CREATE TABLE teacher (  
  number INT,  
  name VARCHAR(100),  
  domain varchar(100),  
  PRIMARY KEY (number)  
) Engine=InnoDB CHARSET=utf8;
```

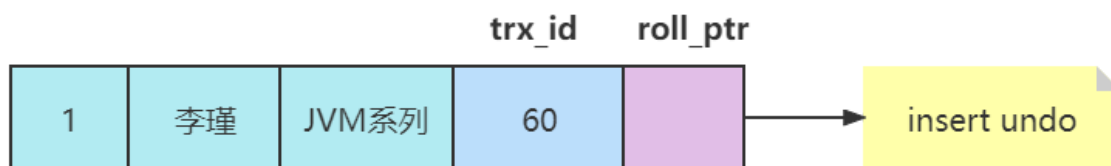
然后向这个表里插入一条数据：

```
INSERT INTO teacher VALUES(1, '李瑾', 'JVM系列');
```

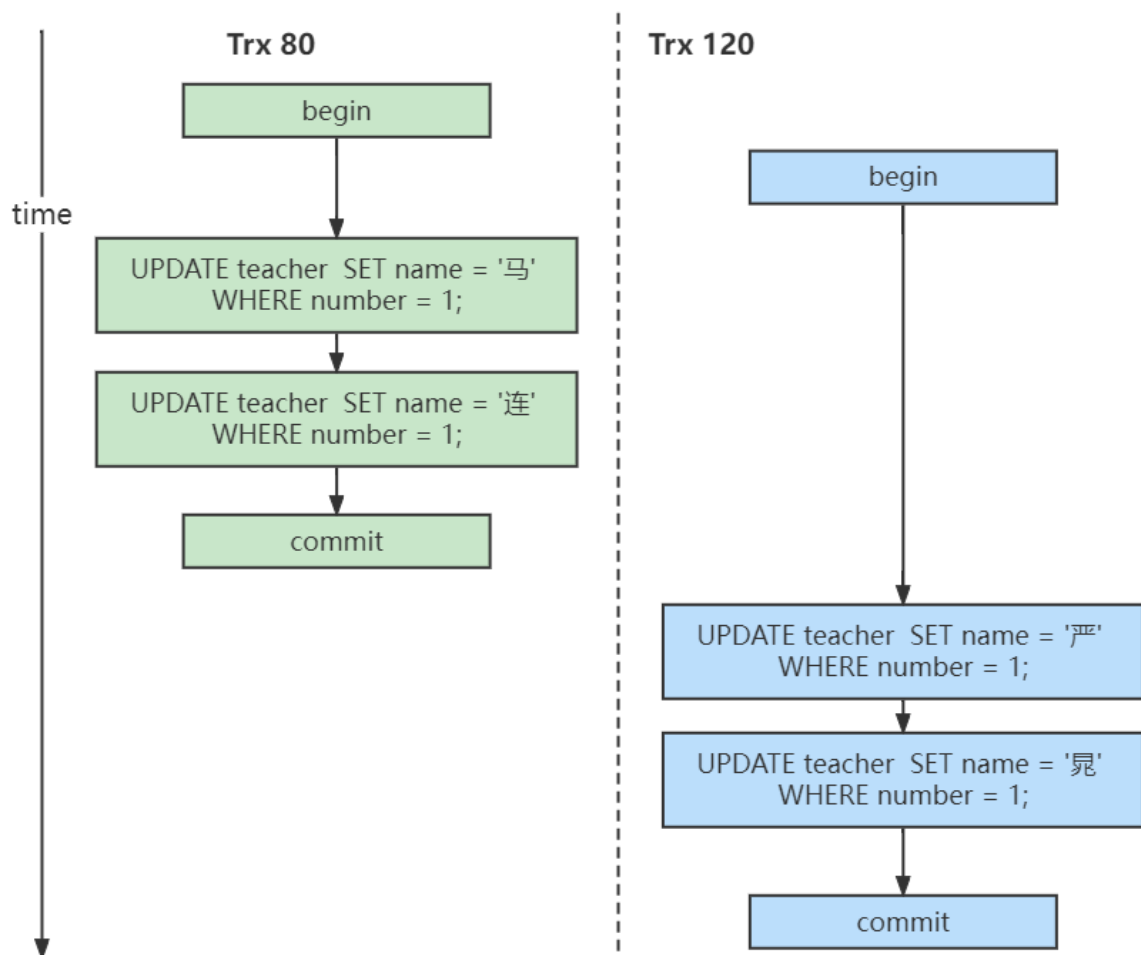
现在表里的数据就是这样的：

number	name	domain
1	李瑾	JVM系列

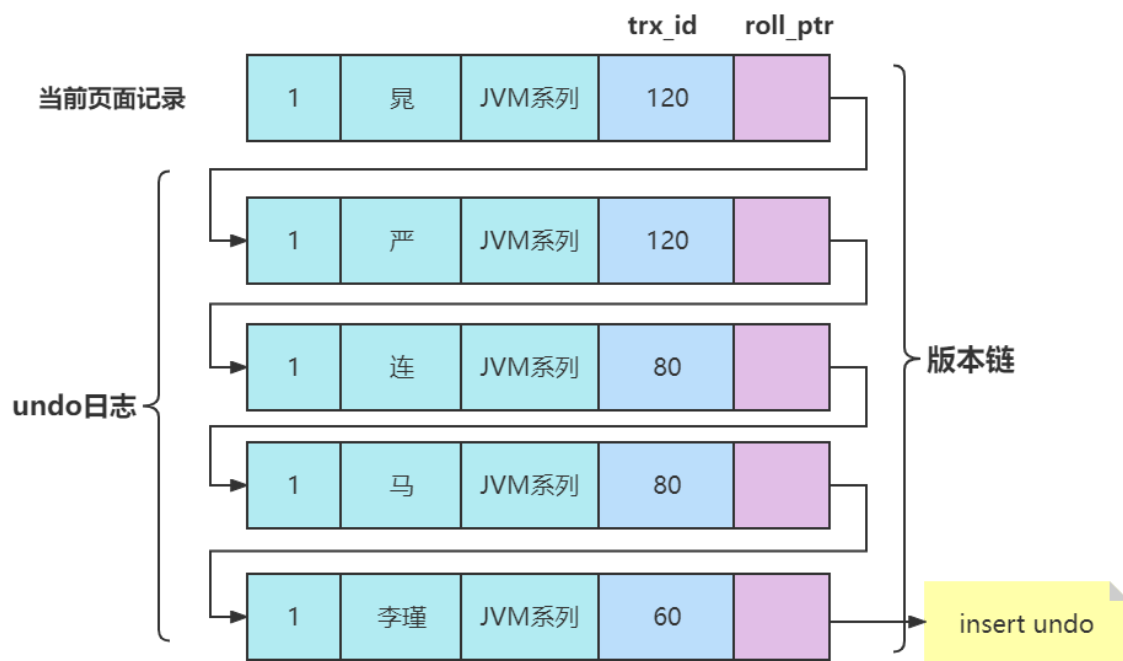
假设插入该记录的事务id为60，那么此刻该条记录的示意图如下所示：



假设之后两个事务id分别为80、120的事务对这条记录进行UPDATE操作，操作流程如下：



每次对记录进行改动，都会记录一条undo日志，每条undo日志也都有一个roll_pointer属性（INSERT操作对应的undo日志没有该属性，因为该记录并没有更早的版本），可以将这些undo日志都连起来，串成一个链表，所以现在的情况就像下图一样：



对该记录每次更新后，都会将旧值放到一条undo日志中，就算是该记录的一个旧版本，随着更新次数的增多，所有的版本都会被roll_pointer属性连接成一个链表，我们把这个链表称之为版本链，版本链的头节点就是当前记录最新的值。另外，每个版本中还包含生成该版本时对应的事务id。**于是可以利用这个记录的版本链来控制并发事务访问相同记录的行为，那么这种机制就被称之为多版本并发控制(Multiversion Concurrency Control MVCC)。**

1.5.1.3.ReadView

必须要知道的概念（作用于SQL查询语句）

对于使用READ UNCOMMITTED隔离级别的事务来说，由于可以读到未提交事务修改过的记录，所以直接读取记录的最新版本就好了（**所以就会出现脏读、不可重复读、幻读**）。

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能

对于使用SERIALIZABLE隔离级别的事务来说，InnoDB使用加锁的方式来访问记录（**也就是所有的事务都是串行的，当然不会出现脏读、不可重复读、幻读**）。

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED 未提交读	可能	可能	可能
READ COMMITTED 已提交读	-	可能	可能
REPEATABLE READ 可重复读	-	-	
SERIALIZABLE 可串行化	-	-	-

对于使用READ COMMITTED和REPEATABLE READ隔离级别的事务来说，都必须保证读到已经提交了的事务修改过的记录，也就是说假如另一个事务已经修改了记录但是尚未提交，是不能直接读取最新版本的记录的，核心问题就是：READ COMMITTED和REPEATABLE READ隔离级别在不可重复读和幻读上的区别是从哪里来的，其实结合前面的知识，这两种隔离级别关键**是需要判断一下版本链中的哪个版本是当前事务可见的。**

为此，InnoDB提出了一个ReadView的概念（作用于SQL查询语句），

这个ReadView中主要包含4个比较重要的内容：

m_ids：表示在生成ReadView时当前系统中活跃的读写事务的事务id列表。

min_trx_id：表示在生成ReadView时当前系统中活跃的读写事务中最小的事务id，也就是m_ids中的最小值。

max_trx_id：表示生成ReadView时系统中应该分配给下一个事务的id值。注意max_trx_id并不是m_ids中的最大值，事务id是递增分配的。比方说现在有id为1，2，3这三个事务，之后id为3的事务提交了。那么一个新的读事务在生成ReadView时，m_ids就包括1和2，min_trx_id的值就是1，max_trx_id的值就是4。

creator_trx_id：表示生成该ReadView的事务的事务id。

1.5.1.4.READ COMMITTED

脏读问题的解决

READ COMMITTED隔离级别的事务在每次查询开始时都会生成一个独立的ReadView。

在MySQL中，READ COMMITTED和REPEATABLE READ隔离级别的一个非常大的区别就是它们生成ReadView的时机不同。

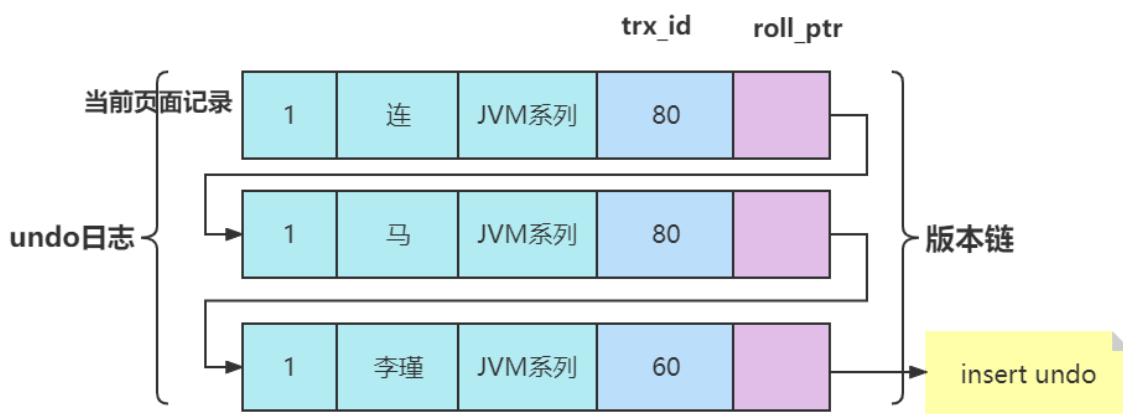
我们还是以表teacher 为例，假设现在表teacher 中只有一条由事务id为60的事务插入的一条记录，接下来来看一下READ COMMITTED和REPEATABLE READ所谓的生成ReadView的时机不同到底不同在哪里。

READ COMMITTED —— 每次读取数据前都生成一个ReadView

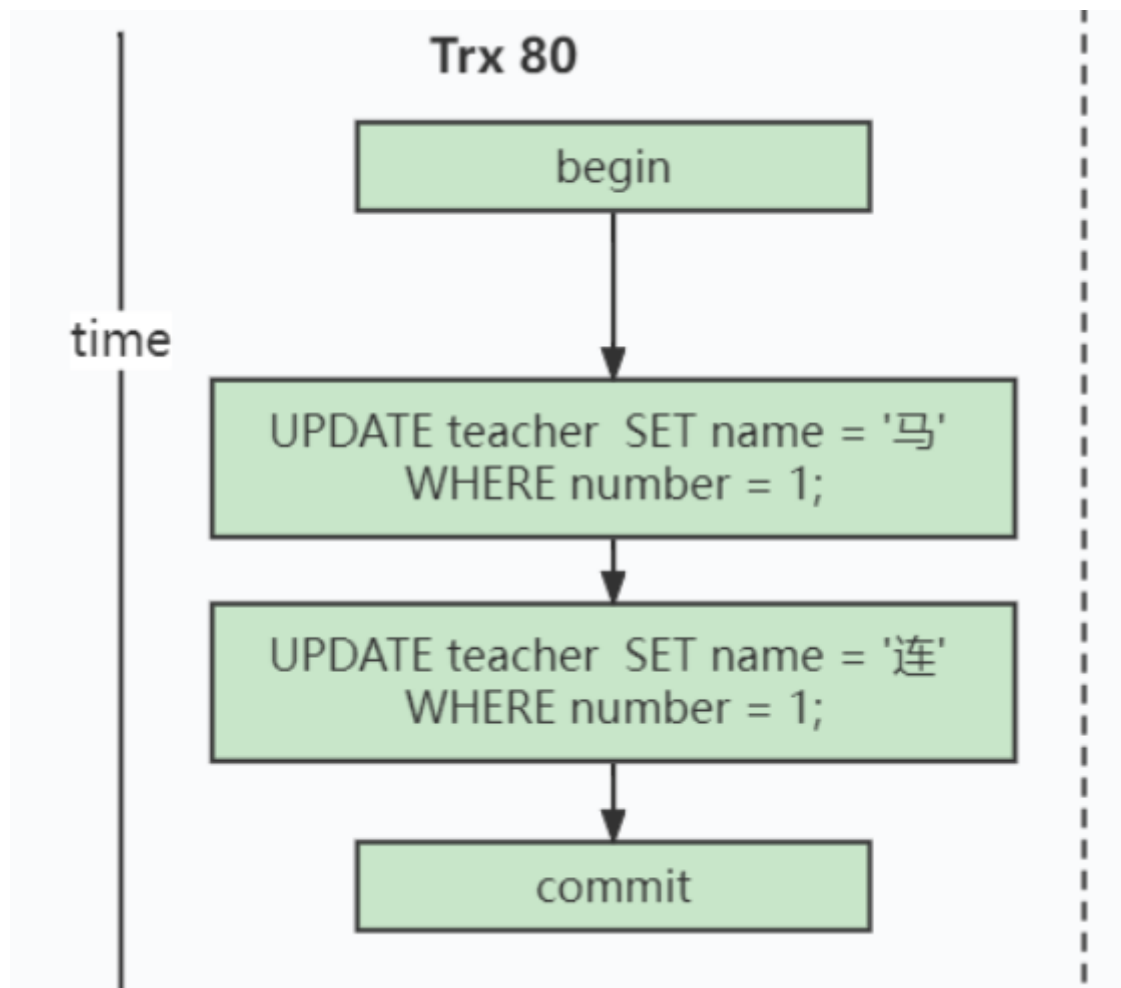
比方说现在系统里有两个事务id分别为80、120的事务在执行：Transaction 80

```
UPDATE teacher SET name = '马' WHERE number = 1;  
UPDATE teacher SET name = '连' WHERE number = 1;  
...
```

此刻，表teacher 中number为1的记录得到的版本链表如下所示：



假设现在有一个使用READ COMMITTED隔离级别的事务开始执行：



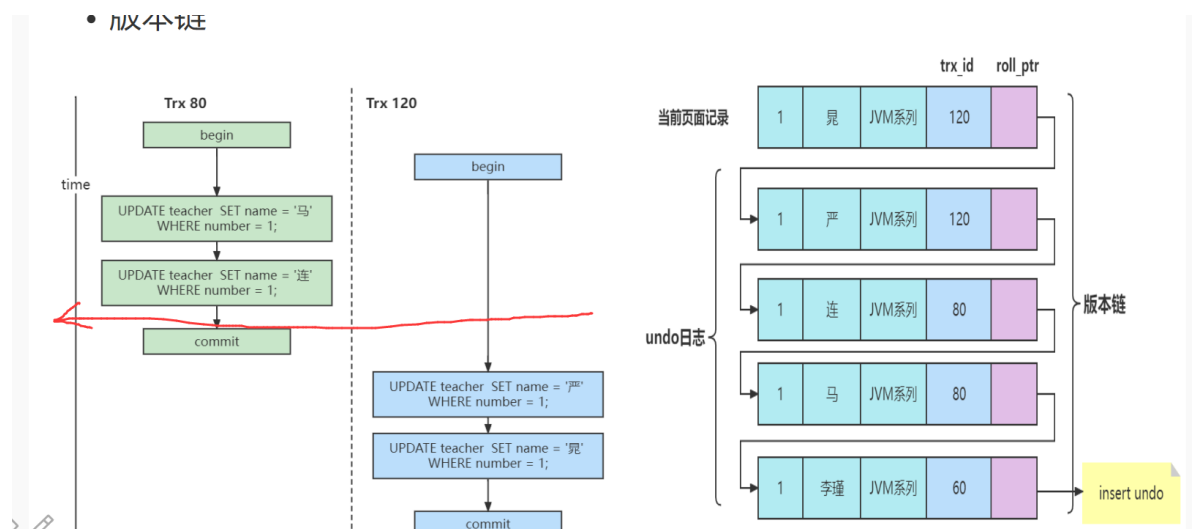
使用READ COMMITTED隔离级别的事务

BEGIN;

SELECE1: Transaction 80、120未提交

SELECT * FROM teacher WHERE number = 1; # 得到的列name的值为'李瑾'

第1次select的时间点 如下图：



这个SELECE1的执行过程如下：

在执行SELECT语句时会先生成一个ReadView：

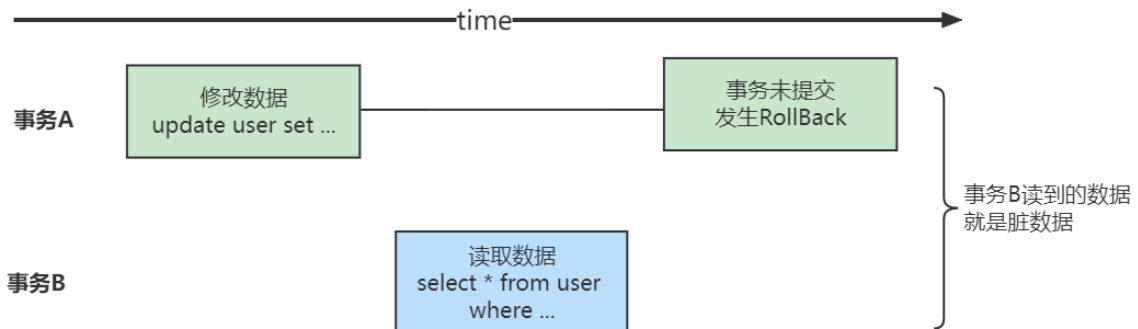
ReadView的m_ids列表的内容就是[80, 120], min_trx_id为80, max_trx_id为121, creator_trx_id为0。

然后从版本链中挑选可见的记录，从图中可以看出，最新版本的列name的内容是'连'，该版本的trx_id值为80，在m_ids列表内，所以不符合可见性要求（trx_id属性值在ReadView的min_trx_id和max_trx_id之间说明创建ReadView时生成该版本的事务还是活跃的，该版本不可以被访问；如果不在，说明创建ReadView时生成该版本的事务已经被提交，该版本可以被访问），根据roll_pointer跳到下一个版本。

下一个版本的列name的内容是'马'，该版本的trx_id值也为80，也在m_ids列表内，所以也不符合要求，继续跳到下一个版本。

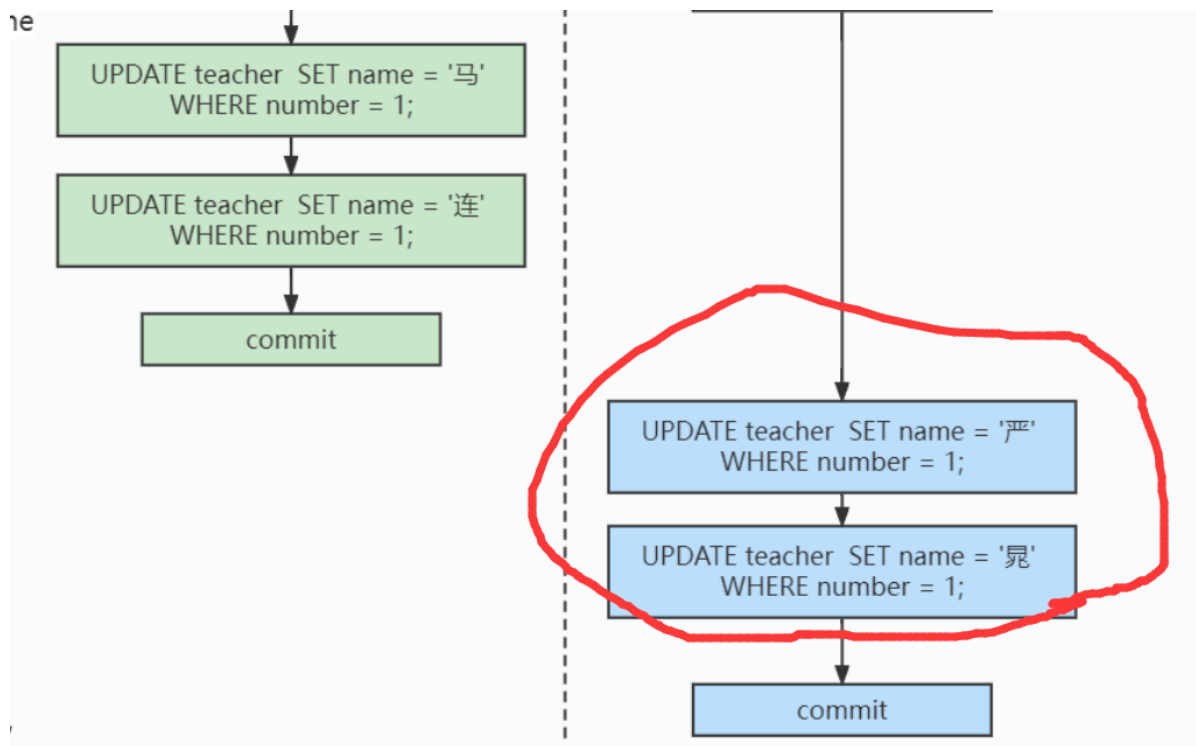
下一个版本的列name的内容是'李瑾'，该版本的trx_id值为60，小于ReadView中的min_trx_id值，所以这个版本是符合要求的，最后返回给用户的版本就是这条列name为'李瑾'的记录。

所以有了这种机制，就不会发生脏读问题！因为会去判断活跃版本，必须是不在活跃版本的才能用，不可能读到没有 commit的记录。



不可重复读问题

然后，我们把事务id为80的事务提交一下，然后再到事务id为120的事务中更新一下表teacher 中 number为1的记录：



Transaction120

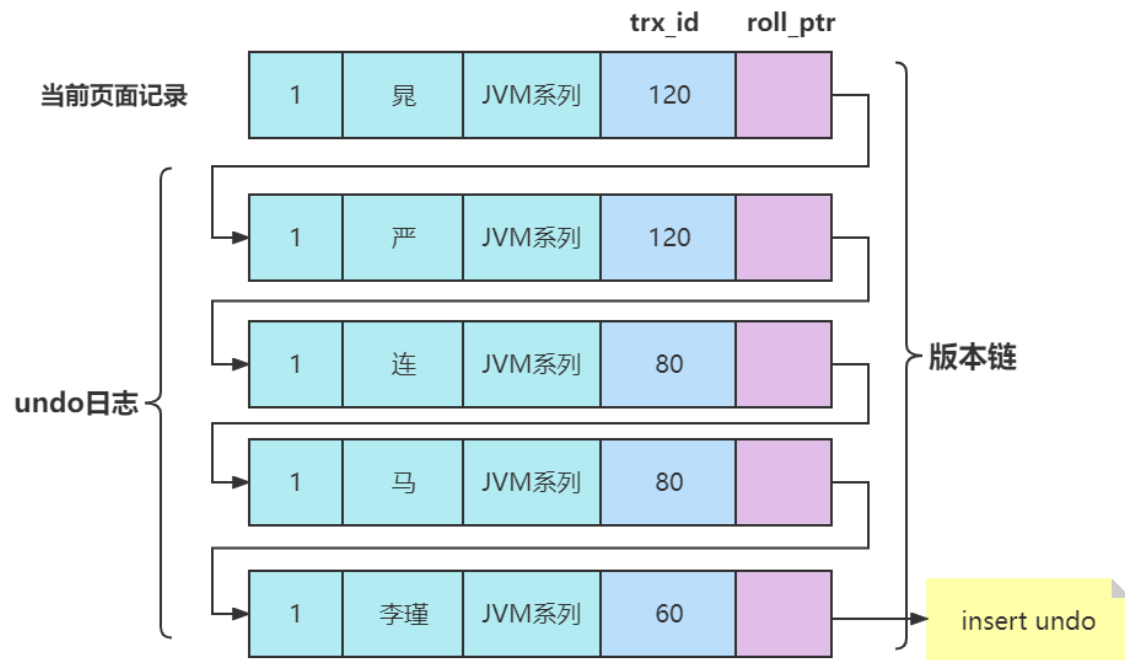
BEGIN;

更新了一些别的表的记录

UPDATE teacher SET name = '严' WHERE number = 1;

UPDATE teacher SET name = '晁' WHERE number = 1;

此刻，表teacher 中number为1的记录版本链就长这样：



然后再到刚才使用READ COMMITTED隔离级别的事务中继续查找这个number为1的记录，如下：

使用READ COMMITTED隔离级别的事务

BEGIN;

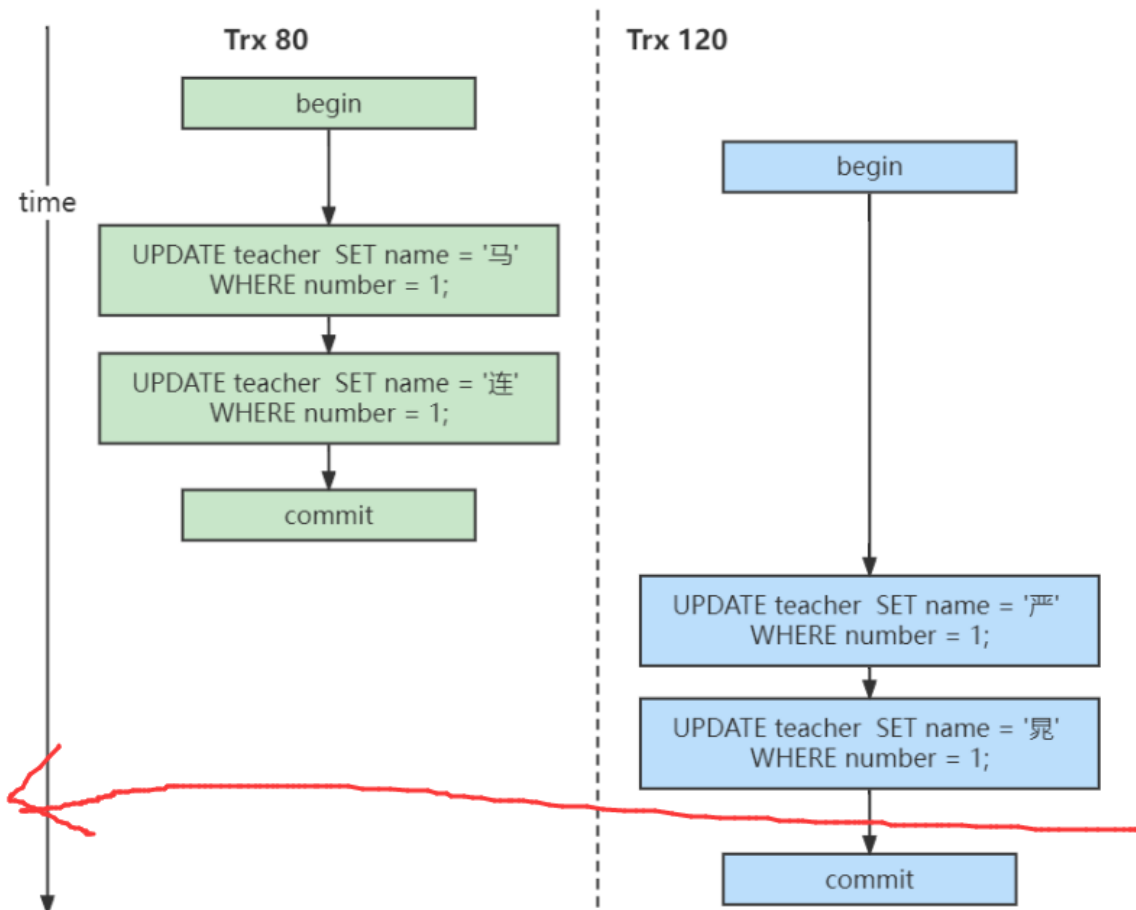
SELECE1: Transaction 80、120均未提交

SELECT * FROM teacher WHERE number = 1; # 得到的列name的值为'李瑾'

SELECE2: Transaction 80提交, Transaction 120未提交

SELECT * FROM teacher WHERE number = 1; # 得到的列name的值为'连'

第2次select的时间点 如下图：



这个SELECT的执行过程如下：

```
SELECT * FROM teacher WHERE number = 1;
```

在执行SELECT语句时会又会单独生成一个ReadView，该ReadView信息如下：

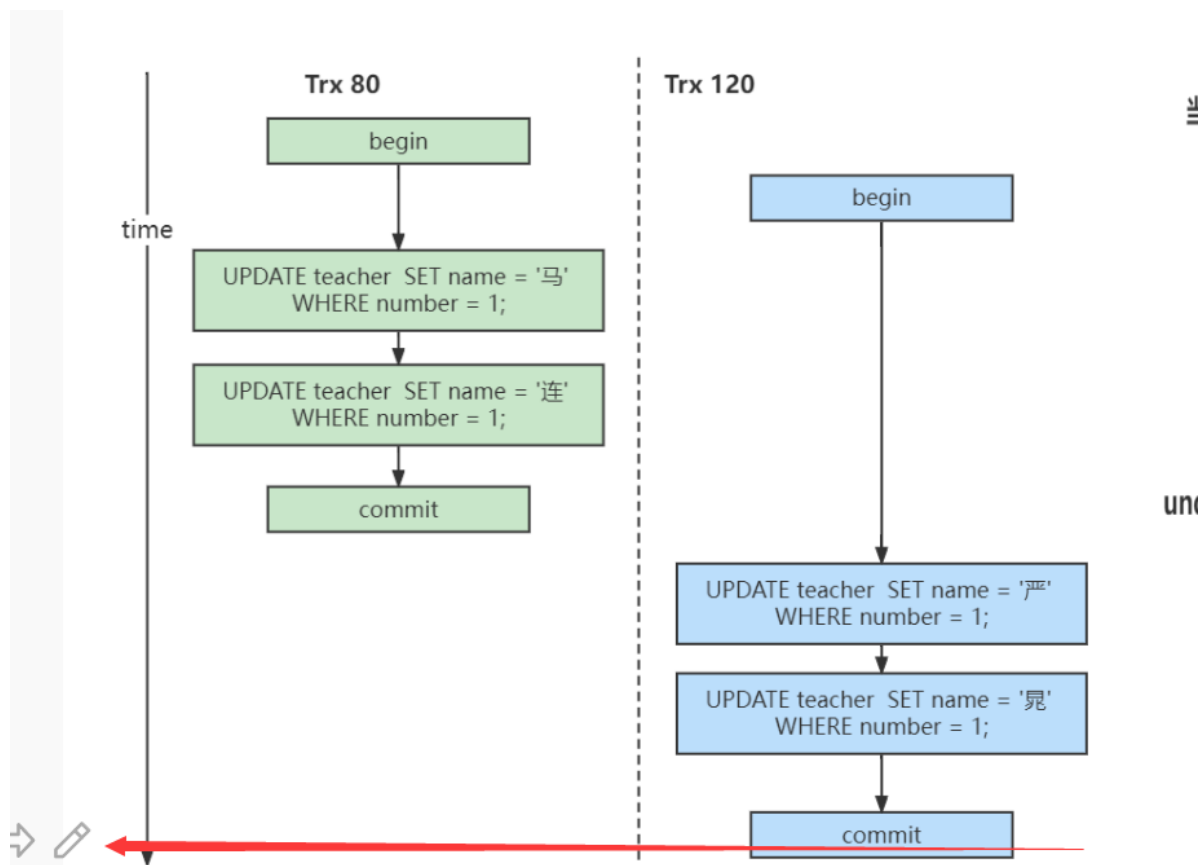
m_ids列表的内容就是[120]（事务id为80的那个事务已经提交了，所以再次生成快照时就没有它了），min_trx_id为120，max_trx_id为121，creator_trx_id为0。

然后从版本链中挑选可见的记录，从图中可以看出，最新版本的列name的内容是'晁'，该版本的trx_id值为120，在m_ids列表内，所以不符合可见性要求，根据roll_pointer跳到下一个版本。

下一个版本的列name的内容是'严'，该版本的trx_id值为120，也在m_ids列表内，所以也不符合要求，继续跳到下一个版本。

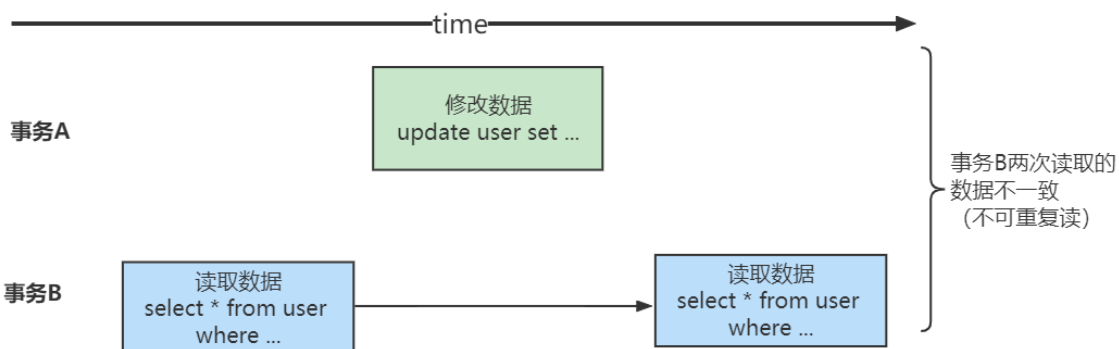
下一个版本的列name的内容是'连'，该版本的trx_id值为80，小于ReadView中的min_trx_id值120，所以这个版本是符合要求的，最后返回给用户的版本就是这条列name为'连'的记录。

以此类推，如果之后事务id为120的记录也提交了，再次在使用READ COMMITTED隔离级别的事务中查询表teacher 中number值为1的记录时，得到的结果就是'晁'了，具体流程我们就不分析了。



但会出现不可重复读问题。

明显上面一个事务中两次



1.5.1.5.REPEATABLE READ

REPEATABLE READ解决不可重复读问题

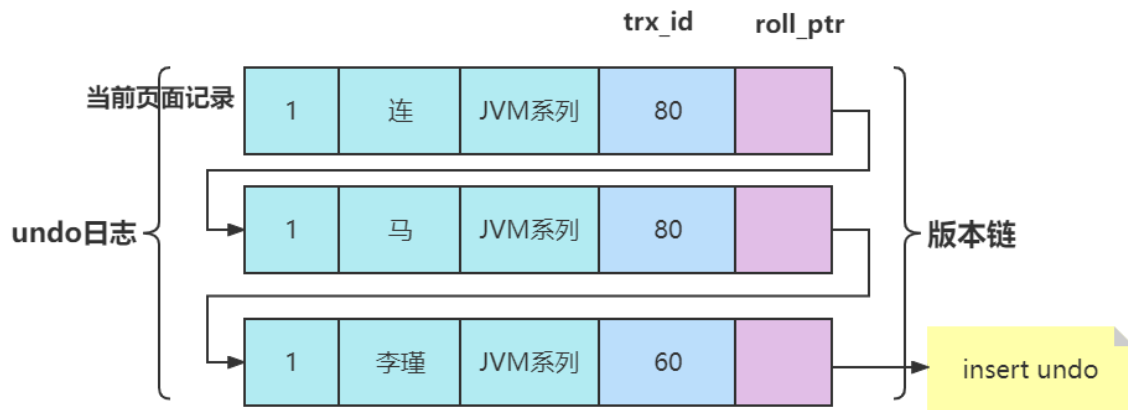
REPEATABLE READ —— 在第一次读取数据时生成一个ReadView

对于使用REPEATABLE READ隔离级别的事务来说，只会在第一次执行查询语句时生成一个ReadView，之后的查询就不会重复生成了。我们还是用例子看一下是什么效果。

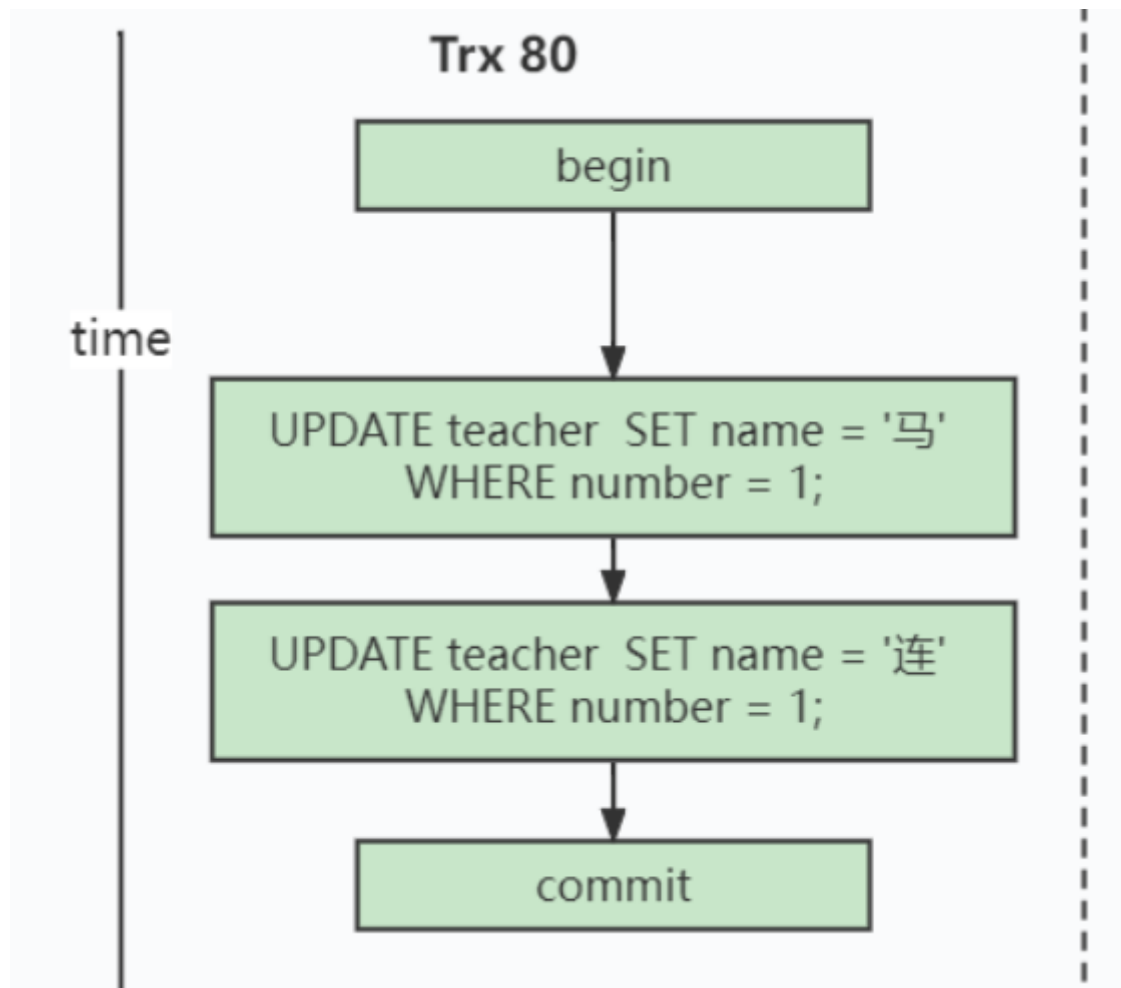
比方说现在系统里有两个事务id分别为80、120的事务在执行：Transaction 80

```
UPDATE teacher SET name = '马' WHERE number = 1;
UPDATE teacher SET name = '连' WHERE number = 1;
...
```

此刻，表teacher 中number为1的记取得到的版本链表如下所示：



假设现在有一个使用REPEATABLE READ隔离级别的事务开始执行：



使用READ COMMITTED隔离级别的事务

```

BEGIN;
SELECE1: Transaction 80、120未提交

SELECT * FROM teacher WHERE number = 1; # 得到的列name的值为'李瑾'
  
```

这个SELECE1的执行过程如下：

在执行SELECT语句时会先生成一个ReadView：

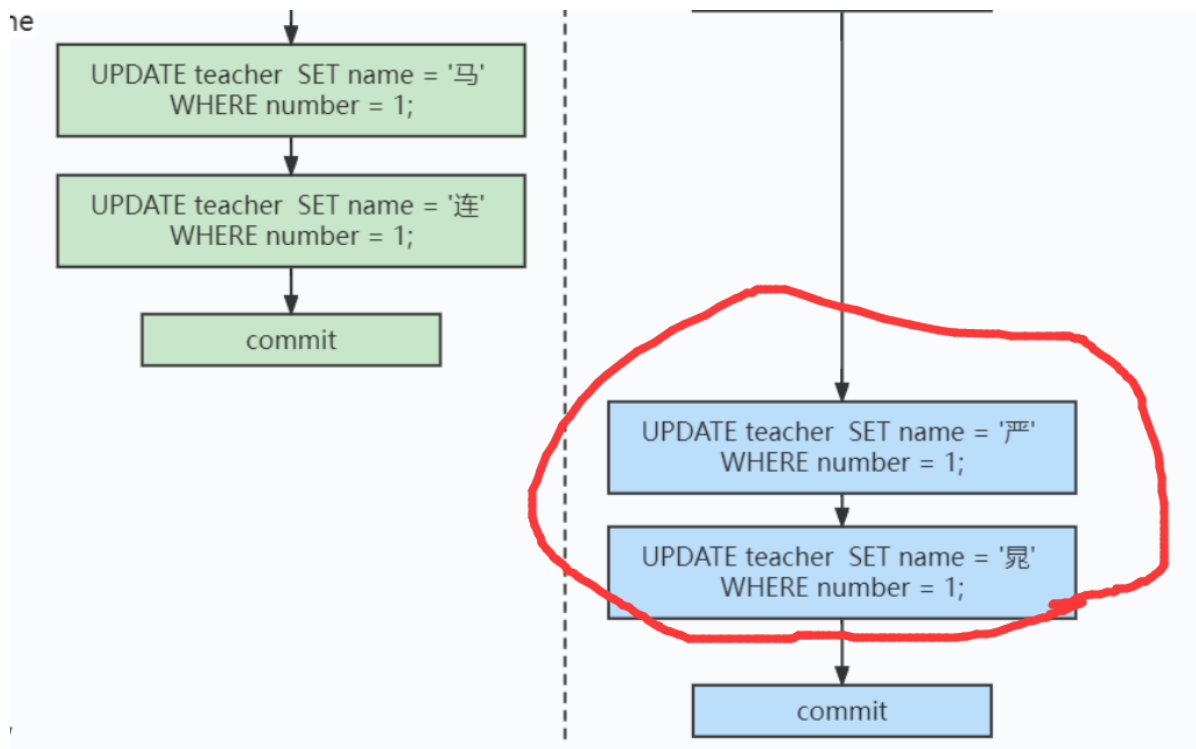
ReadView的m_ids列表的内容就是[80, 120]，min_trx_id为80，max_trx_id为121，creator_trx_id为0。

然后从版本链中挑选可见的记录，从图中可以看出，最新版本的列name的内容是'连'，该版本的trx_id值为80，在m_ids列表内，所以不符合可见性要求（trx_id属性值在ReadView的min_trx_id和max_trx_id之间说明创建ReadView时生成该版本的事务还是活跃的，该版本不可以被访问；如果不在，说明创建ReadView时生成该版本的事务已经被提交，该版本可以被访问），根据roll_pointer跳到下一个版本。

下一个版本的列name的内容是'马'，该版本的trx_id值也为80，也在m_ids列表内，所以也不符合要求，继续跳到下一个版本。

下一个版本的列name的内容是'李瑾'，该版本的trx_id值为60，小于ReadView中的min_trx_id值，所以这个版本是符合要求的，最后返回给用户的版本就是这条列name为'李瑾'的记录。

之后，我们把事务id为80的事务提交一下，然后再到事务id为120的事务中更新一下表teacher 中number为1的记录：



Transaction120

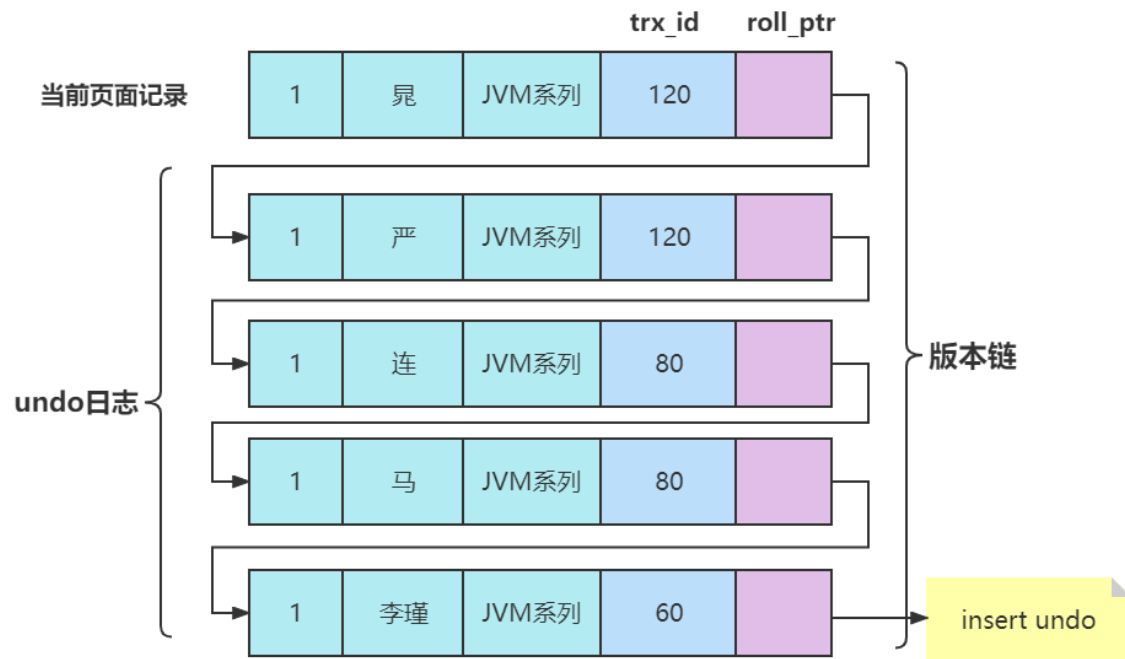
BEGIN;

更新了一些别的表的记录

```
UPDATE teacher SET name = '严' WHERE number = 1;
```

```
UPDATE teacher SET name = '晁' WHERE number = 1;
```

此刻，表teacher 中number为1的记录的版本链就长这样：



然后再到刚才使用REPEATABLE READ隔离级别的事务中继续查找这个number为1的记录，如下：

使用READ COMMITTED隔离级别的事务

```
BEGIN;
```

SELECE1: Transaction 80、120均未提交

```
SELECT * FROM teacher WHERE number = 1; # 得到的列name的值为'李瑾'
```

SELECE2: Transaction 80提交, Transaction 120未提交

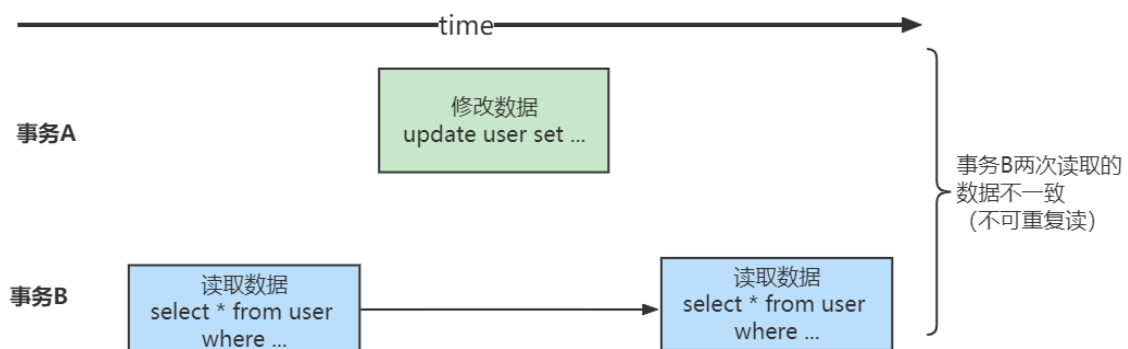
```
SELECT * FROM teacher WHERE number = 1; # 得到的列name的值为'李瑾'
```

这个SELECE2的执行过程如下：

因为当前事务的隔离级别为REPEATABLE READ，而之前在执行SELECE1时已经生成过ReadView了，所以此时直接复用之前的ReadView，之前的ReadView的m_ids列表的内容就是[80, 120]，min_trx_id为80，max_trx_id为121，creator_trx_id为0。

根据前面的分析，返回的值还是'李瑾'。

也就是说两次SELECT查询得到的结果是重复的，记录的列name值都是'李瑾'，这就是可重复读的含义。



总结一下就是：

ReadView中的比较规则(前两条)

- 1、如果被访问版本的trx_id属性值与ReadView中的creator_trx_id值相同，意味着当前事务在访问它自己修改过的记录，所以该版本可以被当前事务访问。
- 2、如果被访问版本的trx_id属性值小于ReadView中的min_trx_id值，表明生成该版本的事务在当前事务生成ReadView前已经提交，所以该版本可以被当前事务访问。

1.5.1.6.MVCC下的幻读解决和幻读现象

前面我们已经知道了，REPEATABLE READ隔离级别下MVCC可以解决不可重复读问题，那么幻读呢？MVCC是怎么解决的？幻读是一个事务按照某个相同条件多次读取记录时，后读取时读到了之前没有读到的记录，而这个记录来自另一个事务添加的新记录。

我们可以想想，在REPEATABLE READ隔离级别下的事务T1先根据某个搜索条件读取到多条记录，然后事务T2插入一条符合相应搜索条件的记录并提交，然后事务T1再根据相同搜索条件执行查询。结果会是什么？按照ReadView中的比较规则(后两条)：

- 3、如果被访问版本的trx_id属性值大于或等于ReadView中的max_trx_id值，表明生成该版本的事务在当前事务生成ReadView后才开启，所以该版本不可以被当前事务访问。
- 4、如果被访问版本的trx_id属性值在ReadView的min_trx_id和max_trx_id之间($\min_trx_id < trx_id < \max_trx_id$)，那就需要判断一下trx_id属性值是不是在m_ids列表中，如果在，说明创建ReadView时生成该版本的事务还是活跃的，该版本不可以被访问；如果不在，说明创建ReadView时生成该版本的事务已经被提交，该版本可以被访问。

不管事务T2比事务T1是否先开启，事务T1都是看不到T2的提交的。请自行按照上面介绍的版本链、ReadView以及判断可见性的规则来分析一下。

但是，在REPEATABLE READ隔离级别下InnoDB中的MVCC可以很大程度地避免幻读现象，而不是完全禁止幻读。怎么回事呢？我们来看下面的情况：

```
mysql> select * from teacher;
+-----+-----+-----+
| number | name  | domain |
+-----+-----+-----+
| 1      | 李瑾  | JVM系列 |
| 2      | 马    | 并发编程 |
| 3      | 连    | MySQL   |
| 4      | 严    | 分布式  |
| 5      | 晁    | 项目实战 |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from teacher where num =30;
ERROR 1054 (42S22): Unknown column 'num' in 'where clause'
mysql> select * from teacher where number =30;
Empty set (0.00 sec)
```

我们首先在事务T1中：

```
select * from teacher where number = 30;
```

很明显，这个时候是找不到number = 30的记录的。
我们在事务T2中，执行：

```
insert into teacher values(30,'豹','数据湖');
```

```
mysql> insert into teacher values(30,'豹','数据湖');  
Query OK, 1 row affected (0.00 sec)
```

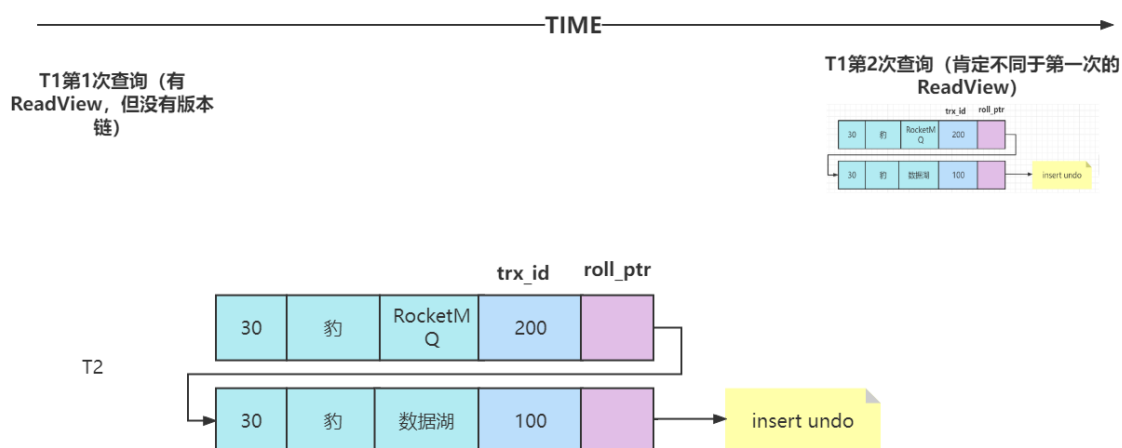
通过执行insert into teacher values(30,'豹','数据湖');，我们往表中插入了一条number = 30的记录。
此时回到事务T1，执行：

```
update teacher set domain='RocketMQ' where number=30;  
select * from teacher where number = 30;
```

```
mysql> update teacher set domain='RocketMQ' where number=30;  
Query OK, 1 row affected (0.00 sec)  
Rows matched: 1  Changed: 1  Warnings: 0  
  
mysql> select * from teacher where number = 30;  
+-----+-----+-----+  
| number | name  | domain |  
+-----+-----+-----+  
|      30 | 豹    | RocketMQ |  
+-----+-----+-----+  
1 row in set (0.00 sec)
```

嗯，怎么回事？事务T1很明显出现了幻读现象。

在REPEATABLE READ隔离级别下，T1第一次执行普通的SELECT 语句时生成了一个ReadView（但是版本链没有），之后T2向teacher 表中新插入一条记录并提交，然后T1也进行了一个update语句。ReadView并不能阻止T1执行UPDATE 或者DELETE 语句来改动这个新插入的记录，但是这样一来，这条新记录的trx_id隐藏列的值就变成了T1的事务id。



之后T1再使用普通的SELECT 语句去查询这条记录时就可以看到这条记录了，也就可以把这条记录返回给客户端。因为这个特殊现象的存在，我们也可以认为MVCC 并不能完全禁止幻读（就是第一次读如果是空的情况，且在自己事务中进行了该条数据的修改）。

1.5.1.7.MVCC小结

从上边的描述中我们可以看出来，所谓的MVCC（Multi-Version Concurrency Control，多版本并发控制）指的就是在使用READ COMMITTD、REPEATABLE READ这两种隔离级别的事务在执行普通的SELECT操作时访问记录的版本链的过程，这样子可以使不同事务的读-写、写-读操作并发执行，从而提升系统性能。

READ COMMITTD、REPEATABLE READ这两个隔离级别的一个很大不同就是：生成ReadView的时机不同，READ COMMITTD在每一次进行普通SELECT操作前都会生成一个ReadView，而REPEATABLE READ只在第一次进行普通SELECT操作前生成一个ReadView，之后的查询操作都重复使用这个ReadView就好了，从而基本上可以避免幻读现象（**就是第一次读如果ReadView是空的情况中的某些情况则避免不了**）。

另外，所谓的MVCC只是在我们进行普通的SEELCT查询时才生效，截止到目前我们所见的所有SELECT语句都算是普通的查询，至于什么是个不普通的查询，后面马上就会讲到（锁定读）。